Sift: a data-preprocessing language

Compiler Construction project, Group 6

Group members

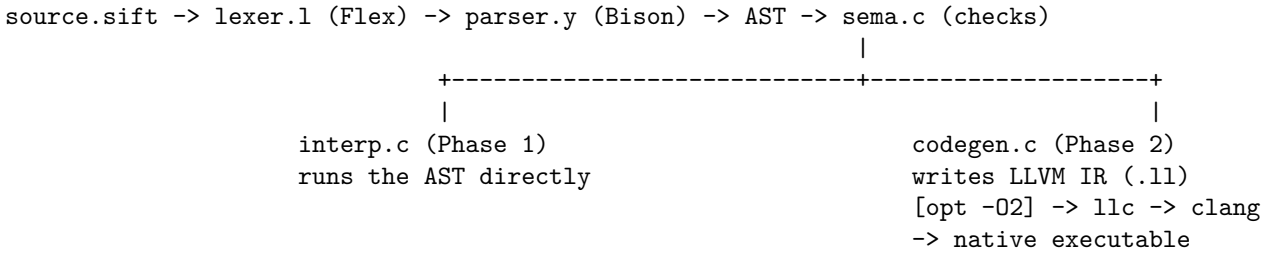
Name	Roll number
Maryam Amanat	23L-0779
Haider Raza	23L-0634
Izen Abbas	23L-0748

What this is

Sift is a small language for cleaning data before machine learning: sorting, filling in missing values and removing outliers. It has a pipeline operator `|>`, a built-in missing value `void`, and range checks with `within`.

The project has two parts that share one front end (lexer, parser, semantic analysis):

- **Phase 1, interpreter** (`./sift run`): built with Flex, Bison and C only. The parser builds an abstract syntax tree (AST), semantic analysis checks it, and a tree-walking interpreter runs it.
- **Phase 2, compiler** (`./sift compile`): the same checked AST is turned into LLVM IR (a `.ll` text file), which `llc` compiles to an object file and `clang` links with a small C runtime into a native executable.



1. Dependencies

Ubuntu (tested on Ubuntu with GCC, Flex 2.6.4, Bison 3.8.2, LLVM/Clang 21). Install everything needed to build, run and test:

```
sudo apt update
sudo apt install build-essential flex bison llvm clang
```

Only if you want to rebuild `README.pdf` from `README.md`:

```
sudo apt install pandoc texlive-latex-recommended texlive-fonts-recommended lmodern
```

Check that the tools are there:

```
gcc --version
make --version
flex --version
```

```
bison --version
llc --version
opt --version
clang --version
```

Any reasonably recent version works. The generated IR uses opaque pointers (`ptr`), so LLVM must be **version 15 or newer** (Ubuntu 24.04 ships LLVM 18). If the LLVM tools have a version suffix on your system (for example `llc-18`), tell the build which ones to use:

```
make test LLC=llc-18 CLANG=clang-18 OPT=opt-18
```

The same names can also be given as environment variables (`LLC`, `CLANG`, `OPT`) when running `./sift compile` directly.

2. How to compile the project

From the project folder:

```
make
```

This runs Bison and Flex, compiles everything with `gcc -std=c11 -Wall -Wextra` (no warnings), and produces:

- `./sift`, the interpreter and compiler driver;
- `build/sift_runtime.o`, the C runtime that is linked into every compiled Sift program.

Bison is run so that any grammar conflict stops the build; the grammar has none. `make clean` removes everything the build produced.

3. How to run it

Phase 1: interpret a program

```
./sift run examples/cleaning.sift
```

Phase 2: compile a program to a native executable

```
./sift compile examples/cleaning.sift -o cleaning
./cleaning
```

Options of `compile`:

Command	What it does
`./sift compile f.sift -o prog`	write LLVM IR, run `llc` and `clang`, produce `prog`
`./sift compile f.sift`	same, the executable is named `f`
`./sift compile f.sift -O -o prog`	also run `opt -O2` on the IR before `llc`
`./sift compile f.sift --emit-ll`	only write the LLVM IR to `f.ll` (or to `-o NAME`)

The value given back by `main` is the program's exit code in both modes (`echo $?` shows it).

Debugging commands

```
./sift tokens examples/sort.sift     # the token stream, one token per line
./sift ast    examples/sort.sift     # the syntax tree after parsing
```

Running the tests

```
make test
```

This builds the project and runs `run_tests.sh`, which runs every test in `tests/` and prints `PASS` or `FAIL` for each, then a summary. Every program test and every error test is run twice: once with `./sift run` and once compiled with `./sift compile` and executed, so both phases must produce exactly the same output and exit code. The script exits with a non-zero status if any test fails. Expected result:

```
212 passed, 0 failed, 212 total
```

The optimised compiler can be tested as well: `./run_tests.sh run compile optimized` (315 checks). `./run_tests.sh run` tests only the interpreter.

Kinds of tests (by file name):

Files	Checked by
`tests/NN_name.sift` + `.expected` (+ optional `.exitcode`)	stdout must equal `.expected`; exit code must equal `.exitcode` (default 0)
`tests/err_NN_name.sift` + `.expected`	must exit non-zero, and stderr must contain the text in `.expected`
`tests/tok_NN_name.sift` + `.expected`	output of `./sift tokens`
`tests/ast_NN_name.sift` + `.expected`	output of `./sift ast`

4. How to use it: a short tour of Sift

A program is a list of **processes** (functions). Execution starts at `process main gives val`. Blocks are written `begin ... end`; comments start with `--`.

Variables, arithmetic and show

```
-- Variables, arithmetic and show
process main gives val
begin
  declare price: val = 120;
  declare qty: val = 3;
  declare paid: flag;
  set price = price - 20;
  show price * qty;
  show 17 / 5;
  show 17 % 5;
  show paid;
  set paid = price * qty >= 300;
  show paid;
  give back 0;
end
```

Output: `300`, `3`, `2`, `no`, `yes` (one per line).

- Types: `val` (32-bit integer), `flag` (`yes` / `no`), `buffer` (array of `val`).
- `declare` creates a variable (`val` starts at 0, `flag` at `no`); `set` changes it.
- `=` inside an expression is always a comparison; there is no `==`.
- Division truncates toward zero. Dividing by zero is a runtime error.

check, else check, otherwise

```
process grade using score: val gives val
```

```
begin
  check score >= 80
  begin
    give back 1;
  end
  else check score >= 50
  begin
    give back 2;
  end
  otherwise
  begin
    give back 3;
  end
end

process main gives val
begin
  show grade(91);
  show grade(64);
  show grade(12);
  give back 0;
end
```

Output: 1, 2, 3.

Loops, stop and skip

```
process main gives val
begin
  declare n: val = 1;
  repeat while n < 100
  begin
    set n = n * 3;
  end
  show n;

  repeat i from 0 below 10
  begin
    check i % 2 = 0
    begin
      skip;
    end
    check i > 6
    begin
      stop;
    end
    show i;
  end

  declare data: buffer = [4, 8, 15];
  declare total: val = 0;
  sweep x in data
  begin
    set total = total + x;
  end
  show total;
```

```
  give back 0;
end
```

Output: `243`, `1`, `3`, `5`, `27`.

- `repeat while c` is a while loop.
- `repeat i from a below b` counts `i = a, a+1, ..., b-1`; `a` and `b` are evaluated once and the body may not `set i`.
- `sweep x in buf` visits every element of a buffer in order (`x` is a read-only copy).
- `stop` leaves the innermost loop (break); `skip` goes to its next iteration (continue).

Processes and recursion

```
process main gives val
begin
  show factorial(5);
  give back 0;
end

-- processes may be used before they are defined
process factorial using n: val gives val
begin
  check n <= 1
  begin
    give back 1;
  end
  give back n * factorial(n - 1);
end
```

Output: `120`.

A process with `gives T` must end with `give back` (or with a `check ... otherwise` whose every branch ends with `give back`). A process without `gives` returns with `give back;` or by reaching `end`. `val` and `flag` arguments are copied; buffers are shared (see below).

Buffers

```
process double_all using d: buffer
begin
  repeat i from 0 below count d
  begin
    set d[i] = d[i] * 2;
  end
end

process main gives val
begin
  declare a: buffer[3];
  declare b: buffer = [1, 2, 3];
  show count a;
  show a[0];
  double_all(b);
  show b[2];
  declare c: buffer = b;
  set c[0] = 100;
  show b[0];
  give back 0;
end
```

Output: `3`, `0`, `6`, `100`.

- `buffer[3]` has 3 zeros; `buffer[4] = [..]` needs exactly 4 values; `buffer = [..]` takes its size from the list; `buffer = other` shares another buffer.
- `count b` is the length.
- Buffers have **reference semantics**: passing one to a process, or copying it with `set` or `declare`, shares the same elements.
- An index outside the buffer is a runtime error.

Missing values and range checks

```
process main gives val
begin
  declare temps: buffer = [21, void, 19, 85];
  sweep t in temps
  begin
    check t is void
    begin
      show void;
    end
    else check t within 0..40
    begin
      show t;
    end
    otherwise
    begin
      show no;
    end
  end
  give back 0;
end
```

Output: `21`, `void`, `19`, `no`.

`void` is a `val` that means "missing" and `show` prints it as `void`. `x within a..b` is `a <= x and x <= b` (inclusive), with `x` computed once. `and` and `or` short-circuit.

The pipeline operator

`a |> f(x, y)` means `f(a, x, y)`, and steps chain from left to right:

```
process add using a: val, b: val gives val
begin
  give back a + b;
end

process square using a: val gives val
begin
  give back a * a;
end

process main gives val
begin
  show 3 |> square |> add(1);
  show add(square(3), 1);
  give back 0;
end
```

Output: `10`, `10` (the two lines mean the same thing).

The full cleaning pipeline (`examples/cleaning.sift`)

This example marks readings outside 0..100 as missing, fills every missing value with the mean of the good readings, and sorts the result:

```
process main gives val
begin
  declare readings: buffer[8] = [12, void, 97, 15, 300, 14, void, 11];
  set readings = readings
    |> mark_outliers(0, 100)
    |> fill_missing
    |> sort_buffer;
  sweep x in readings
  begin
    show x;
  end
  give back 0;
end
```

(`mark_outliers`, `mean_of`, `fill_missing` and `sort_buffer` are defined in the same file.)

```
$ ./sift run examples/cleaning.sift
11
12
14
15
29
29
29
97
```

300 is outside 0..100 and becomes `void`; the good values 12+97+15+14+11 = 149, and 149/5 = 29; the three missing slots become 29; then the buffer is sorted. The compiled program prints the same.

Errors

Errors are printed as `file.sift:LINE: error: message` with exit code 1. A lexical or syntax error stops at the first one; semantic errors are all reported before stopping:

```
process main gives val
begin
  declare x: val = yes;
  show y;
  stop;
  give back 0;
end
```

```
e1.sift:3: error: initial value must be val, not flag
e1.sift:4: error: 'y' is not declared
e1.sift:5: error: 'stop' used outside a loop
```

Runtime errors (in both modes) print `runtime error: line N: message` and exit with code 1:

```
process main gives val
begin
  declare d: buffer = [1, 2, 3];
  show d[1];
  show d[3];
```

```
  give back 0;
end
```

```
2
runtime error: line 5: index 3 out of bounds for buffer of length 3
```

What the generated LLVM IR looks like

`./sift compile t4.sift --emit-ll` for the factorial example writes (in part):

```
define i32 @sift.factorial(i32 %a.n) {
entry:
  %v0.n = alloca i32
  store i32 %a.n, ptr %v0.n
  %t1 = load i32, ptr %v0.n
  %t2 = icmp sle i32 %t1, 1
  br i1 %t2, label %L2, label %L3
L2:
  ret i32 1
...
L1:
  %t3 = load i32, ptr %v0.n
  %t4 = load i32, ptr %v0.n
  %t5 = sub i32 %t4, 1
  %t6 = call i32 @sift.factorial(i32 %t5)
  %t7 = mul i32 %t3, %t6
  ret i32 %t7
...
}
```

Variables live in `alloca` stack slots (`opt -O2` turns them into registers). The Sift `main` becomes `@sift_main`, which the C `main` in the runtime calls; other processes are named `@sift.NAME`.

5. Every file in the project

File	What it does
`Makefile`	builds `./sift` and `build/sift_runtime.o`; `make test`, `make clean`
`run_tests.sh`	runs all tests in both modes, prints PASS/FAIL and a summary
`README.md` / `README.pdf`	this document (the PDF is made with `pandoc README.md -o README.pdf`)
`.gitignore`	keeps build outputs, compiled programs and `snapshots/` out of git
`docs/EXPLANATION.md`	plain-language guide to how the compiler works, for the viva
`docs/Sift_Proposal_Group6.pdf`	the original project proposal
`src/lexer.l`	Flex specification: keywords, identifiers, integers, operators, comments, line numbers
`src/lexer.h`	declarations for using the lexer (`yylex`, `yylineno`, `token_name`)
`src/parser.y`	Bison grammar with precedence declarations; its actions build the AST and desugar the pipeline
`src/ast.h`, `src/ast.c`	AST node types, constructors, pipeline desugaring, and the `./sift ast` printer

File	What it does
src/sema.h, src/sema.c	semantic analysis: scopes and symbol tables, type checking, all semantic rules, variable slot numbers
src/interp.h, src/interp.c	Phase 1: tree-walking interpreter and runtime errors
src/codegen.h, src/codegen.c	Phase 2: walks the AST and prints textual LLVM IR
src/runtime.c	C runtime linked into compiled programs: show, runtime errors, buffer storage, C main
src/main.c	command-line driver: run, compile (calls opt, llc, clang), tokens, ast
src/util.h, src/util.c	error reporting, checked malloc, a growable list type
examples/cleaning.sift	the full cleaning pipeline from the proposal
examples/sort.sift	insertion sort from the proposal, with a main
tests/	109 test programs (see section 3) and their expected output

Files created by the build (not part of the source): sift, build/ (generated parser.c, parser.h, lexer.c, object files and sift_runtime.o).

6. Design decisions and known limitations

- **No floating point.** val is a 32-bit signed integer; the mean in the cleaning example is an integer mean (149 / 5 = 29).
- **void is a sentinel.** A missing value is stored as -2147483648 (INT_MIN), so that number cannot be used as real data: show prints it as void and is void is true for it. Arithmetic on void is not checked. An integer literal must fit in 0 .. 2147483647 (negative numbers come from unary minus).
- **Integer overflow wraps around** (two's complement) in both modes. INT_MIN / -1 gives INT_MIN and INT_MIN % -1 gives 0 instead of crashing.
- **Buffers** are never freed (programs are short-lived), so a process can give back a buffer it declared. A buffer must have at least one element. A sized declaration (buffer[N]) can only be initialised with a list; declare d: buffer = other; shares other.
- **Scopes:** every begin ... end block is a scope. Parameters share the scope of the top level of the process body, so the body cannot redeclare a parameter. A loop variable is in its own scope around the loop body. A declaration's initial value is evaluated before the new name exists.
- **Evaluation order** is left to right everywhere (operands, arguments, list elements). For set d[i] = v the index is evaluated and bounds-checked before v.
- **"Must end with give back"** is checked conservatively, as specified: the last statement must be give back, or a check ... otherwise in which every branch ends that way. So a gives process cannot end with a loop, even an endless one.
- **Exit codes** are taken modulo 256 by Linux (main giving back 300 exits with 44).
- **Deep recursion:** the interpreter runs on a thread with a 1 GiB stack (tested to a recursion depth of 200000). Extremely deep recursion can still overflow the stack, in either mode, without a friendly message.
- **Syntax errors** are Bison's messages (e.g. syntax error, unexpected give); only the first one is reported.
- **The grammar** uses Bison precedence declarations, so it also accepts not not x, - -x and a |> f > 0 (meaning f(a) > 0); all of these are still type-checked.
- **LLVM IR** uses opaque pointers (ptr), so a buffer is { ptr, i32 }; LLVM 15+ is required. Runtime checks in compiled code are an inline compare and a branch to an error block that calls a noreturn runtime function, so the division or memory access is only reached when it is safe.
- **Compiled programs** need build/sift_runtime.o, which ./sift finds next to itself (SIFT_RUNTIME=path overrides this). Temporary .ll and .o files go in /tmp/sift-XXXXXX and are deleted afterwards.